Dimensionality reduction

CSCI-P 556

ZORAN TIGANJ

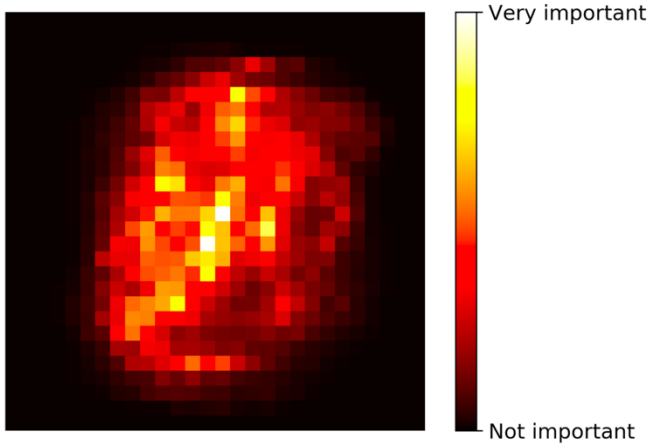
Today

- Dimensionality reduction
  - Main Approaches for Dimensionality Reduction
    - Projection
    - Manifold learning
  - Two popular algorithms for dimensionality reduction:
    - PCA (kernel PCA)
    - LLE
- Slides are based on Chapter 8 from the Hands on Machine Learning Textbook.

- Why dimensionality reduction?

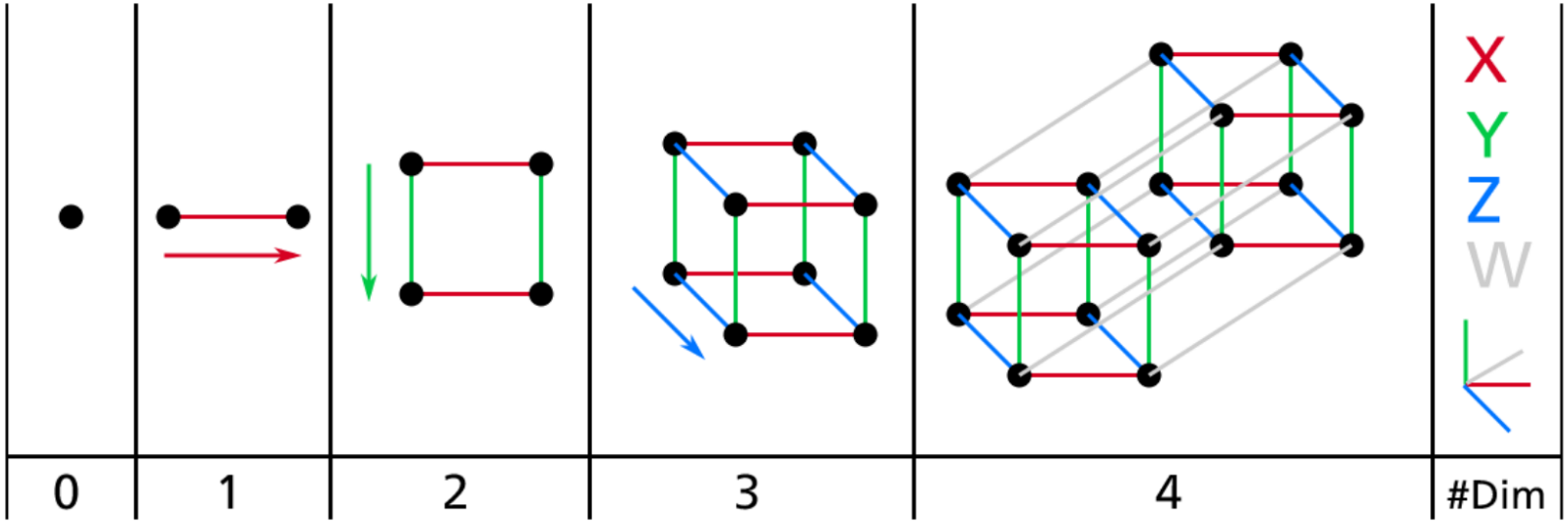
Dimensionality reduction

- Many Machine Learning problems involve thousands or even millions of features for each training instance.
  - This makes training slow
  - This also makes it harder to find a good solution
- It is often possible to reduce the number of features considerably, turning an intractable problem into a tractable one.
- Dimensionality reduction is also extremely useful for data visualization:
  - Reducing the number of dimensions down to two (or three) makes it possible to plot a condensed view of a high-dimensional training set.



The Curse of Dimensionality

- Let's start by imagining a 200-dimensional ellipsoid bent in a 1,000-dimensional space.
- How about 4D cube unfolded in 3D?

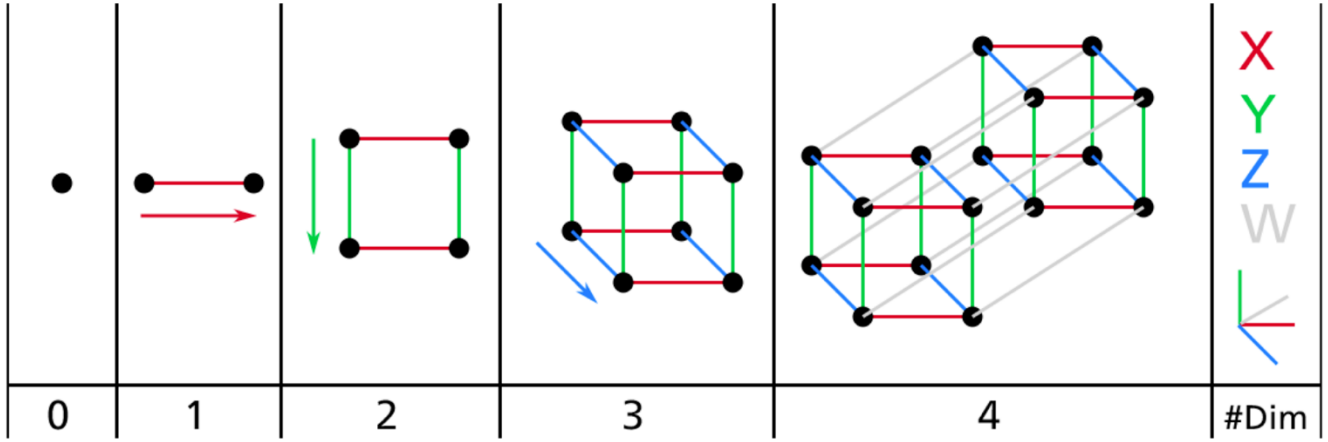


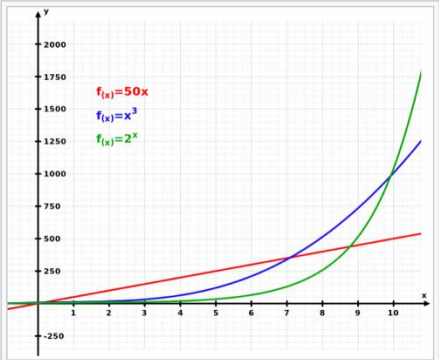
The Curse of Dimensionality

- If you pick two points randomly in a unit square, the distance between these two points will be, on average, roughly 0.52.
- If you pick two random points in a unit 3D cube, the average distance will be roughly 0.66.
- But what about two points picked random randomly in a 1,000,000-dimensional hypercube?
  - The average distance will be about 408.25 - there's plenty of space in high dimensions.
  - As a result, high-dimensional datasets are at risk of being very sparse: most training instances are likely to be far away from each other.
  - This also means that a new instance will likely be far away from any training instance.
- Can we solve this by increasing the number of training instances?

The Curse of Dimensionality

- Unfortunately, in practice, the number of training instances required to reach a given density grows exponentially with the number of dimensions.
  - With just 100 features (significantly fewer than in the MNIST problem), you would need more training instances than atoms in the observable universe in order for training instances to be within 0.1 of each other on average, assuming they were spread out uniformly across all dimensions.





The graph illustrates how exponential growth (green) surpasses both linear (red) and cubic (blue) growth.

Linear growth
Cubic growth
Exponential growth

Wikipedia

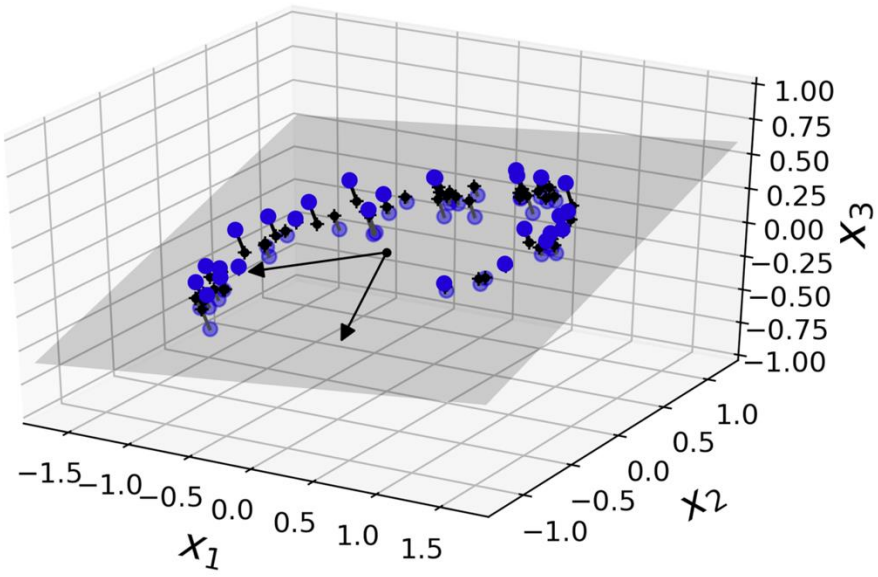
Main Approaches for Dimensionality Reduction

- Projection
- Manifold Learning

Projection

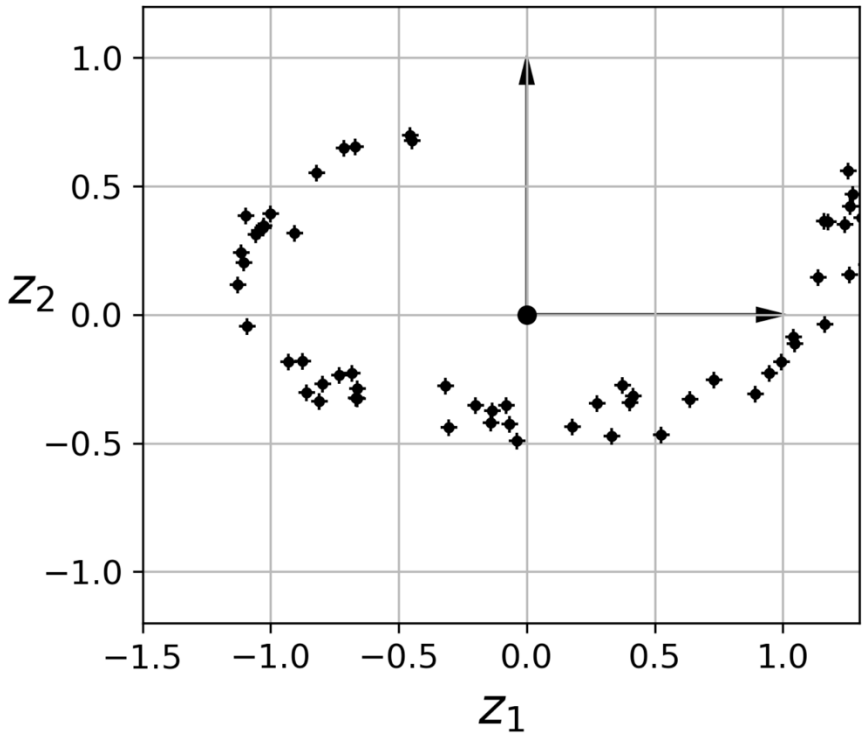
- In most real-world problems, training instances are not spread out uniformly across all dimensions:
  - Many features are almost constant, while others are highly correlated.
  - As a result, all training instances lie within (or close to) a much lower-dimensional subspace of the high-dimensional space.

A 3D dataset lying close to a 2D subspace



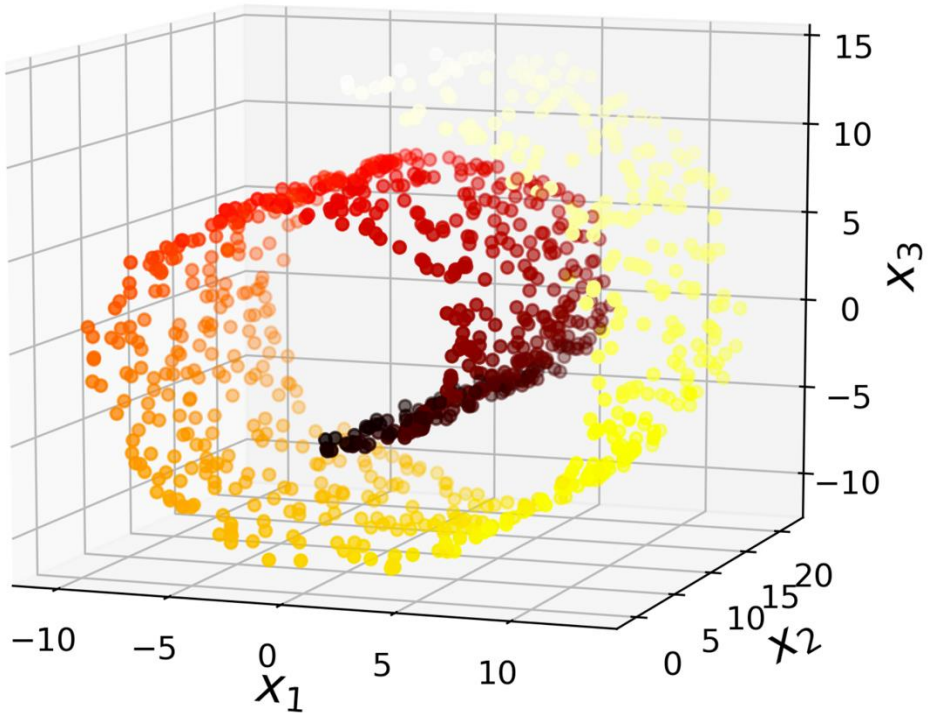
Projection

- If we project every training instance perpendicularly onto this subspace we get a new 2D dataset:

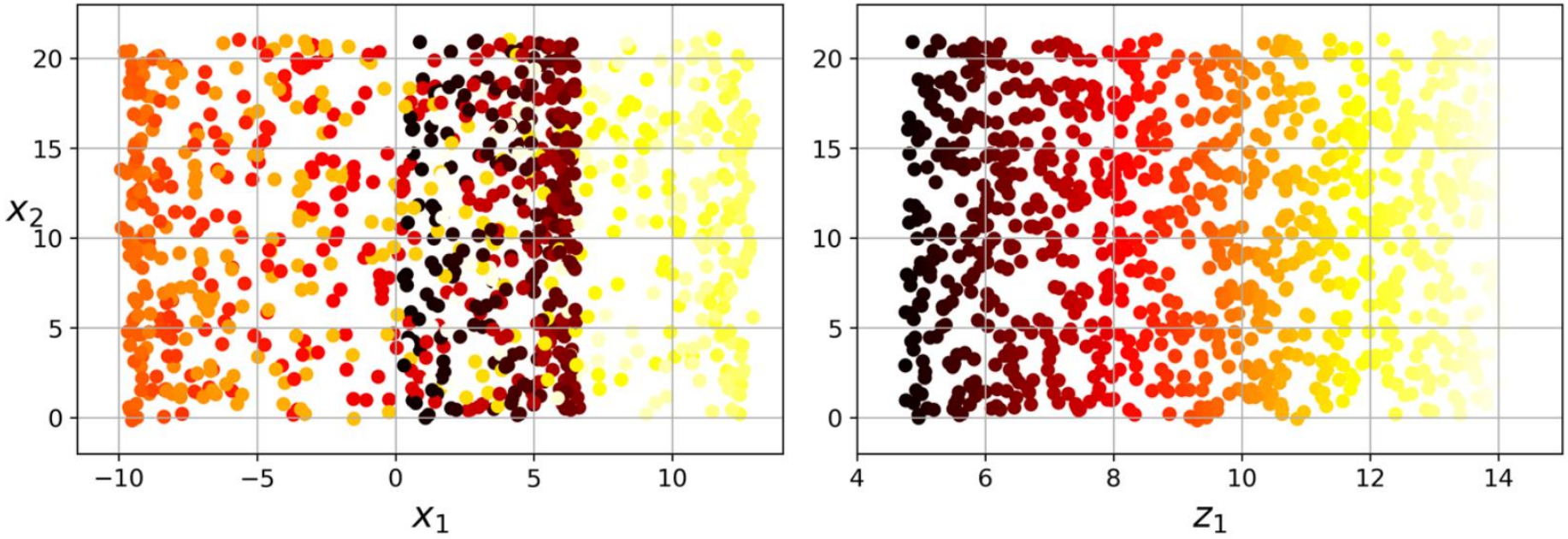


Projection

- However, projection is not always the best approach to dimensionality reduction. In many cases the subspace may twist and turn, such as in the famous Swiss roll toy dataset



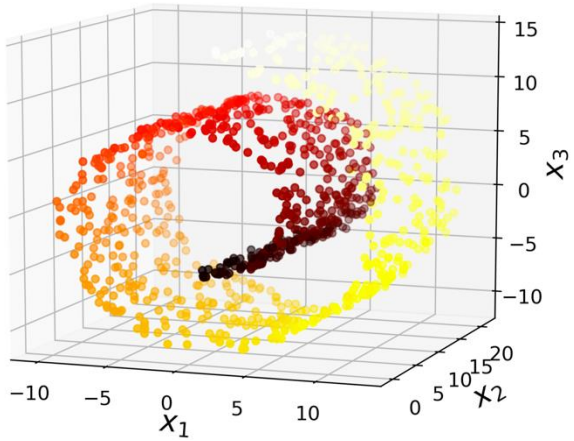
Projection

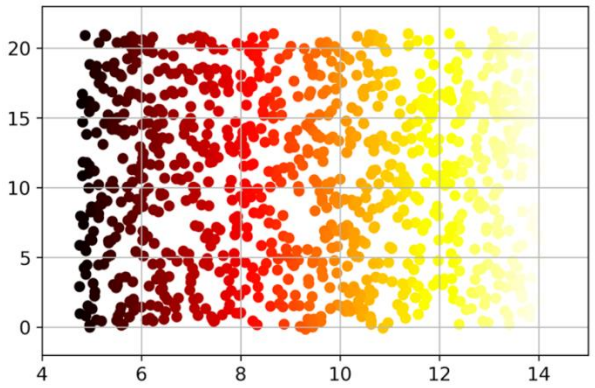


Squashing by projecting onto a plane (left) versus unrolling the Swiss roll (right)

Manifold Learning

- The Swiss roll is an example of a 2D manifold: a 2D manifold is a 2D shape that can be bent and twisted in a higher-dimensional space.
- More generally, a d-dimensional manifold is a part of an n-dimensional space (where d < n) that locally resembles a d-dimensional hyperplane.
  - In the case of the Swiss roll, d = 2 and n = 3: it locally resembles a 2D plane, but it is rolled in the third dimension.



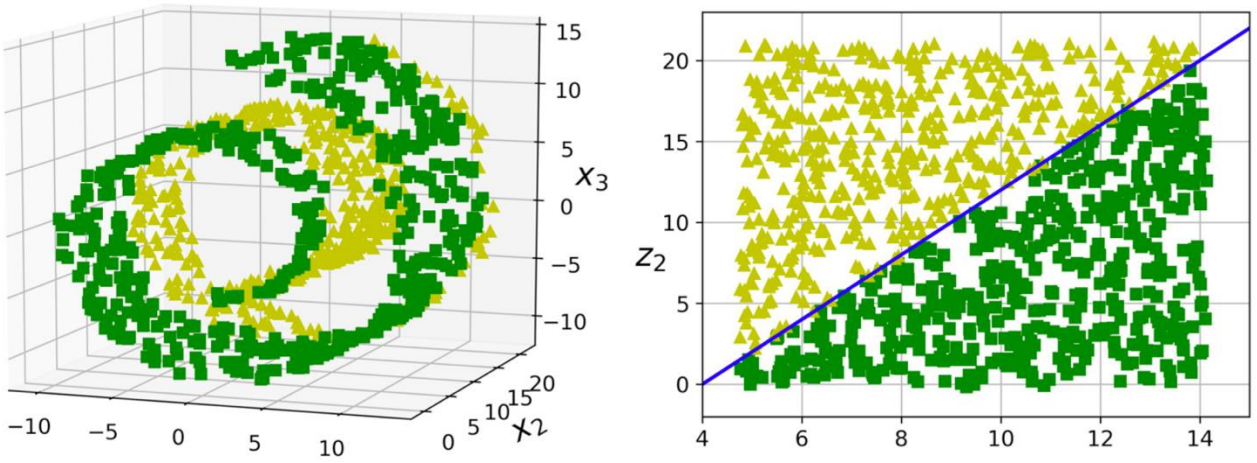


Manifold Learning

- Many dimensionality reduction algorithms work by modeling the manifold on which the training instances lie; this is called Manifold Learning.
- Manifold learning assumes that most real-world high-dimensional datasets lie close to a much lower-dimensional manifold.
- Example: MNIST
  - All handwritten digit images have some similarities.
  - They are made of connected lines, the borders are white, and they are more or less centered.
  - If you randomly generated images, only a ridiculously tiny fraction of them would look like handwritten digits.
  - So MNIST dataset can be squeezed into a lower dimensional manifold.

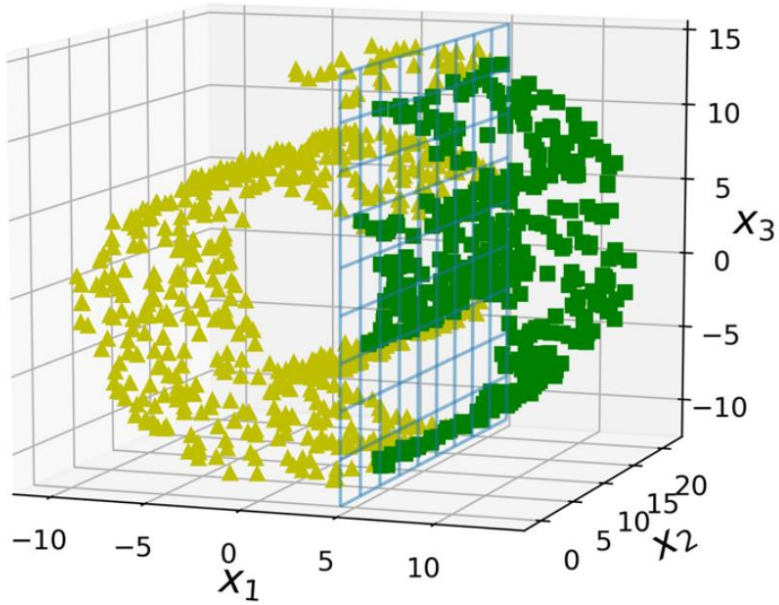
Manifold Learning

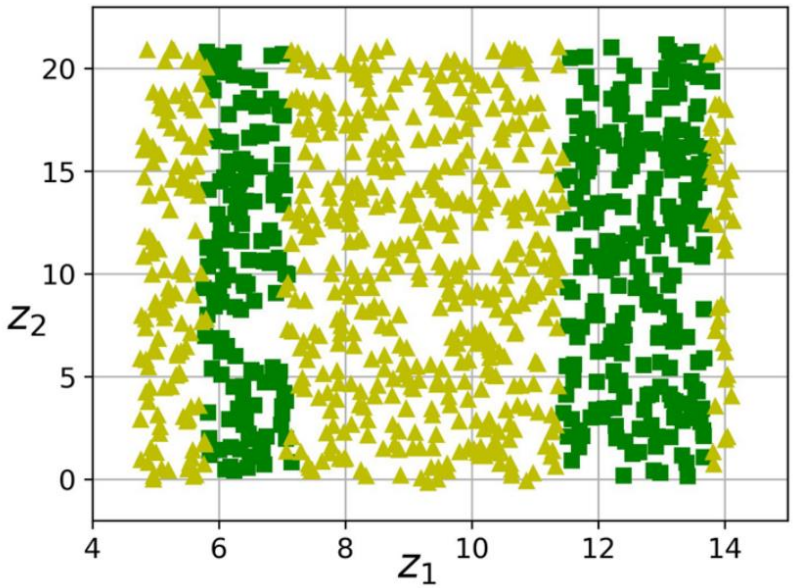
- The manifold assumption is often accompanied by another implicit assumption: that the task at hand (e.g., classification or regression) will be simpler if expressed in the lower-dimensional space of the manifold.



Manifold Learning

- However, this implicit assumption does not always hold



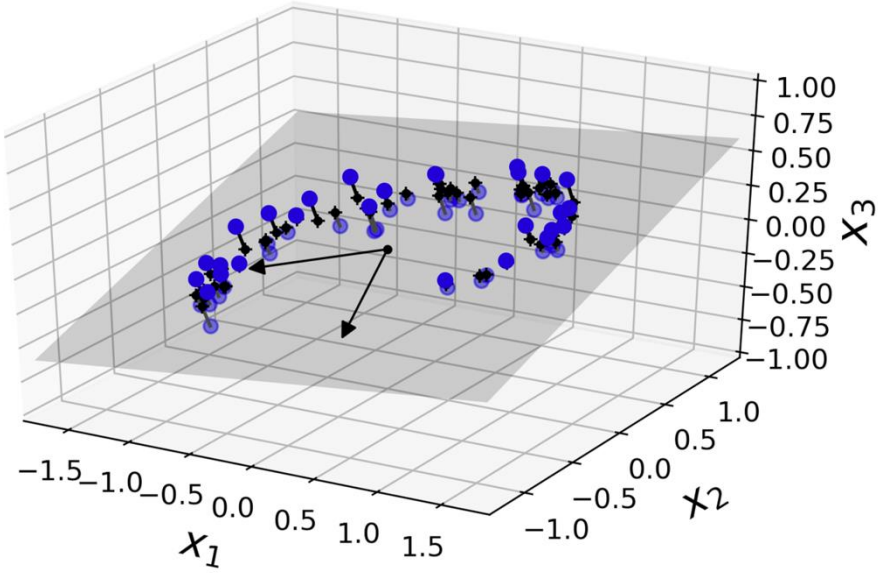


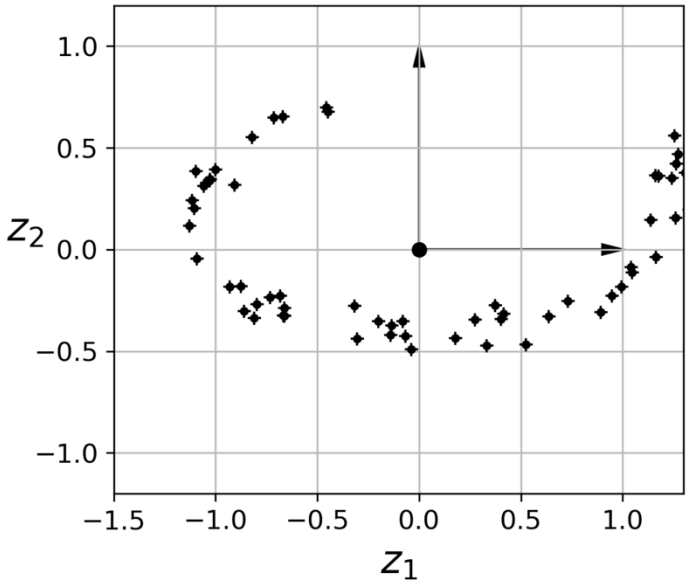
We will cover two popular algorithms for dimensionality reduction

- Principal Component Analysis (PCA)
- Locally Linear Embedding (LLE)

Principal Component Analysis (PCA)

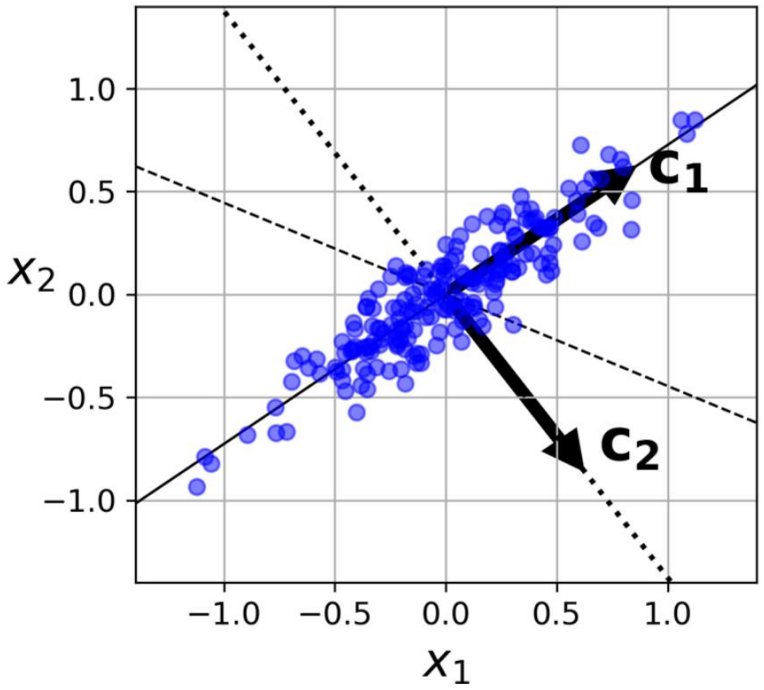
- Principal Component Analysis (PCA) is by far the most popular dimensionality reduction algorithm.
- First it identifies the hyperplane that lies closest to the data, and then it projects the data onto it,





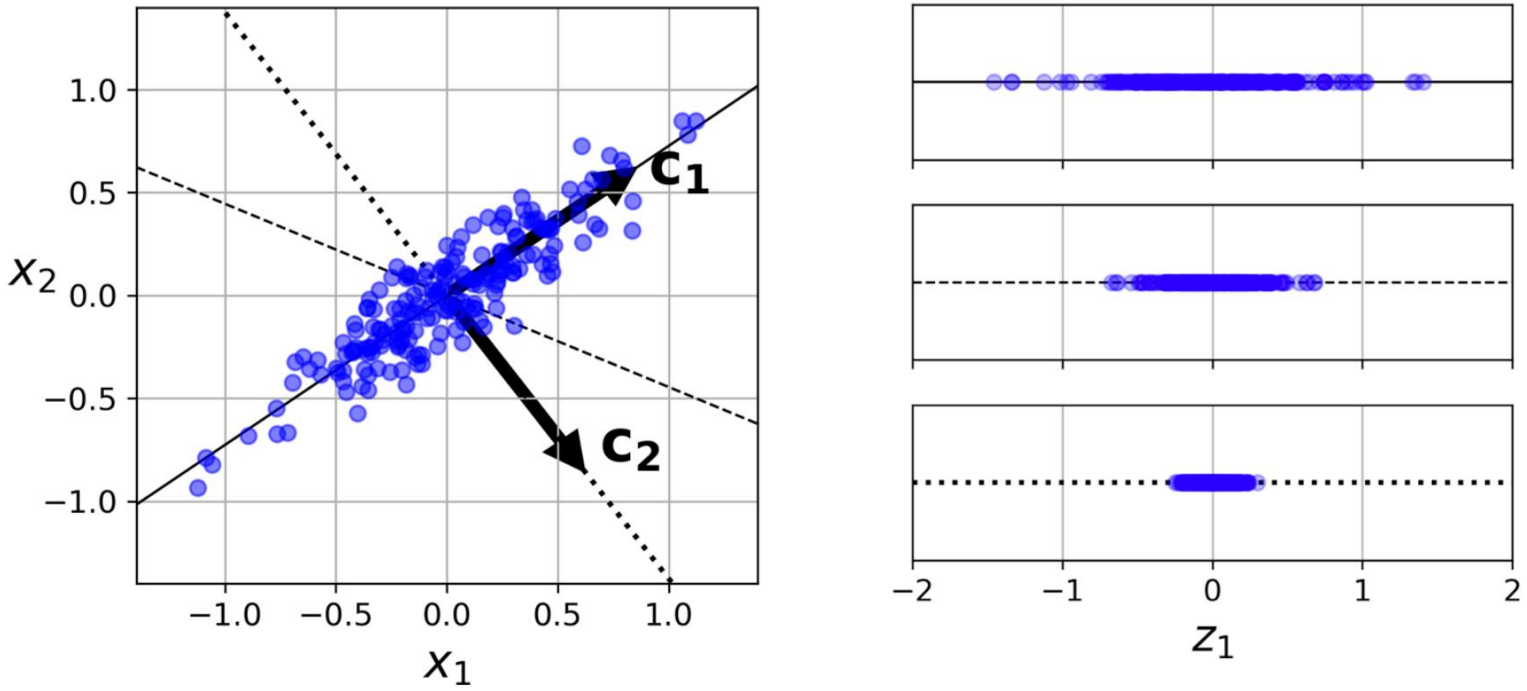
Preserving the Variance

- Before you can project the training set onto a lower-dimensional hyperplane, you first need to choose the right hyperplane.



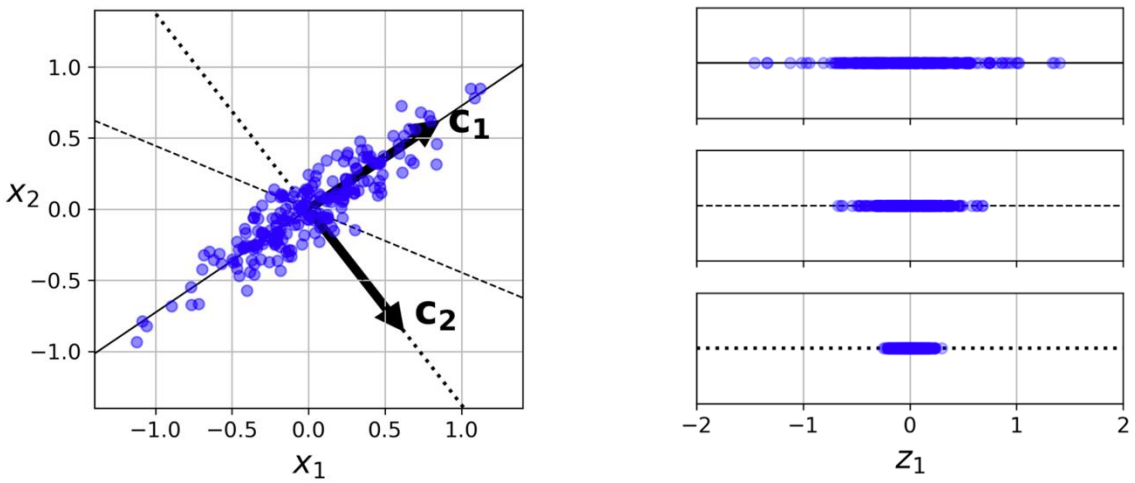
Preserving the Variance

- Before you can project the training set onto a lower-dimensional hyperplane, you first need to choose the right hyperplane.



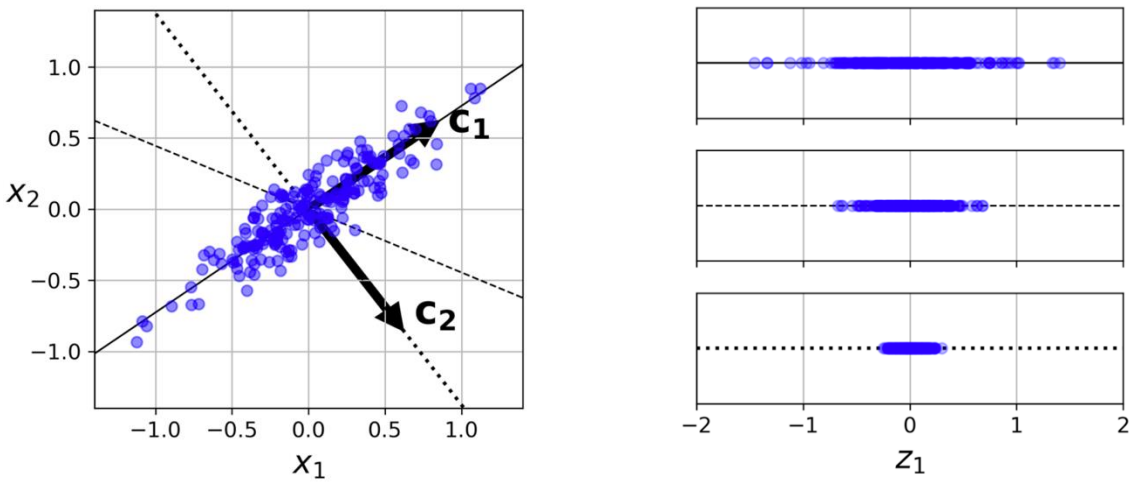
Preserving the Variance

- It seems reasonable to select the axis that preserves the maximum amount of variance, as it will most likely lose less information than the other projections.
- Another way to justify this choice is that it is the axis that minimizes the mean squared distance between the original dataset and its projection onto that axis.



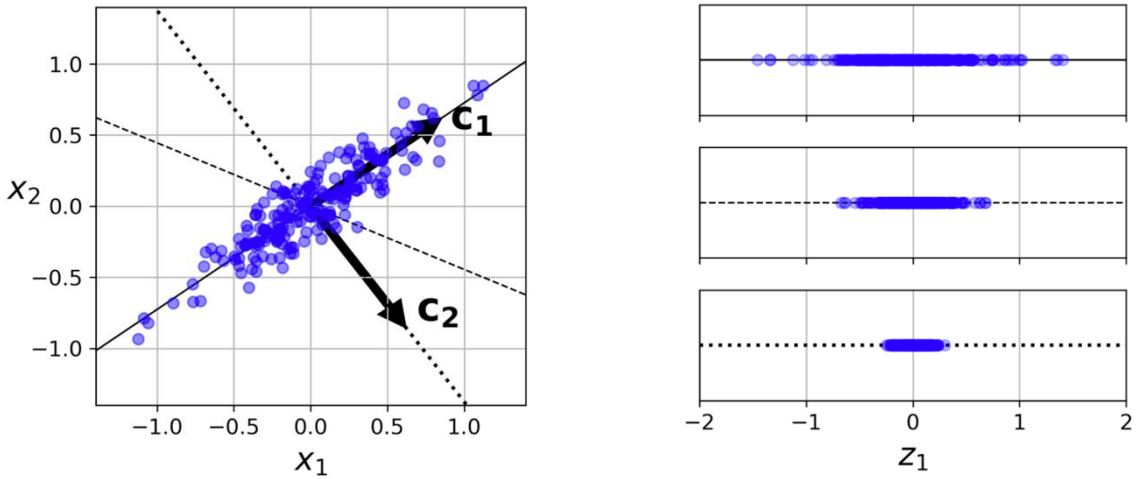
Principal Components

- PCA identifies the axis that accounts for the largest amount of variance in the training set.
- It also finds a second axis, orthogonal to the first one, that accounts for the largest amount of remaining variance.



Principal Components

- PCA identifies the axis that accounts for the largest amount of variance in the training set.
- It also finds a second axis, orthogonal to the first one, that accounts for the largest amount of remaining variance.



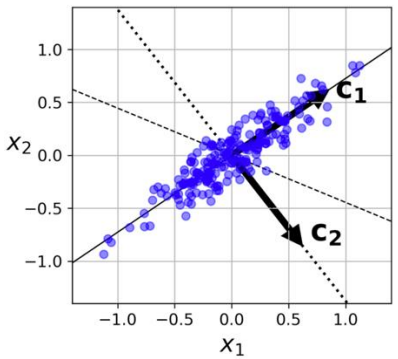
In this 2D example: it is the dotted line (principal components are orthogonal).

Principal Components

- If it were a higher-dimensional dataset, PCA would also find a third axis, orthogonal to both previous axes, and a fourth, a fifth, and so on—a s many axes as the number of dimensions in the dataset.

Finding principal components

- So how can you find the principal components of a training set?
- There is a standard matrix factorization technique called Singular Value Decomposition (SVD) that can decompose the training set matrix **X** into the matrix multiplication of three matrices $\mathbf{U}\,\mathbf{\Sigma}\,\mathbf{V}^T$, where **V** contains the unit vectors that define all the principal components that we are looking for



$$\mathbf{V} = \begin{pmatrix} | & | & & | \\ \mathbf{c}_1 & \mathbf{c}_2 & \cdots & \mathbf{c}_n \\ | & | & & | \end{pmatrix}$$

```
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)
c1 = Vt.T[:, 0]
c2 = Vt.T[:, 1]
```

SVD example

Consider the 4×5 matrix

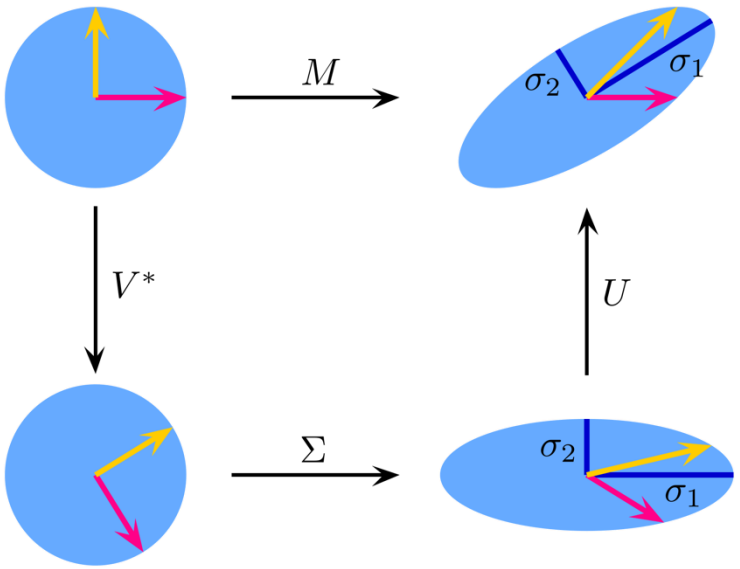
$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 & 2 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 \end{bmatrix}$$

A singular value decomposition of this matrix is given by $\mathbf{U\Sigma V}^*$

$$\mathbf{U} = \begin{bmatrix} 0 & -1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

$$\mathbf{\Sigma} = \begin{bmatrix} 3 & 0 & 0 & 0 & 0 \\ 0 & \sqrt{5} & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & \mathbf{0} & 0 \end{bmatrix}$$

$$\mathbf{V}^* = \begin{bmatrix} 0 & 0 & -1 & 0 & 0 \\ -\sqrt{0.2} & 0 & 0 & 0 & -\sqrt{0.8} \\ 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ -\sqrt{0.8} & 0 & 0 & 0 & \sqrt{0.2} \end{bmatrix}$$



$$M = U \cdot \Sigma \cdot V^*$$

Wikipedia

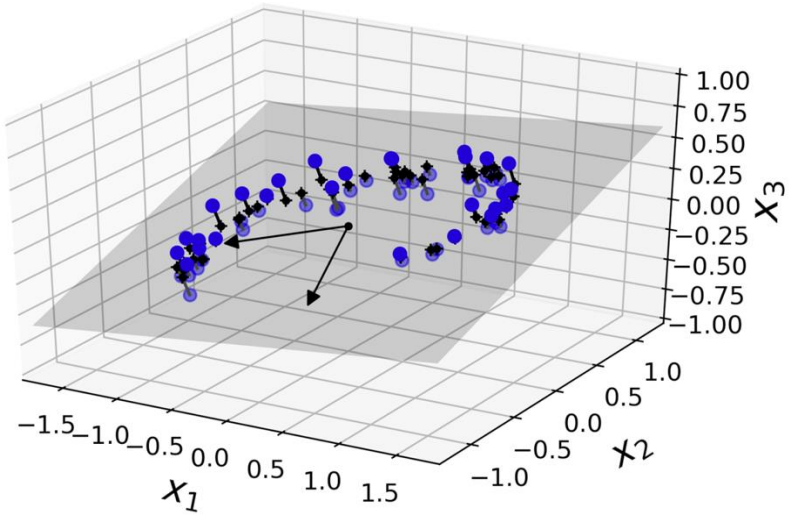
Projecting Down to d Dimensions

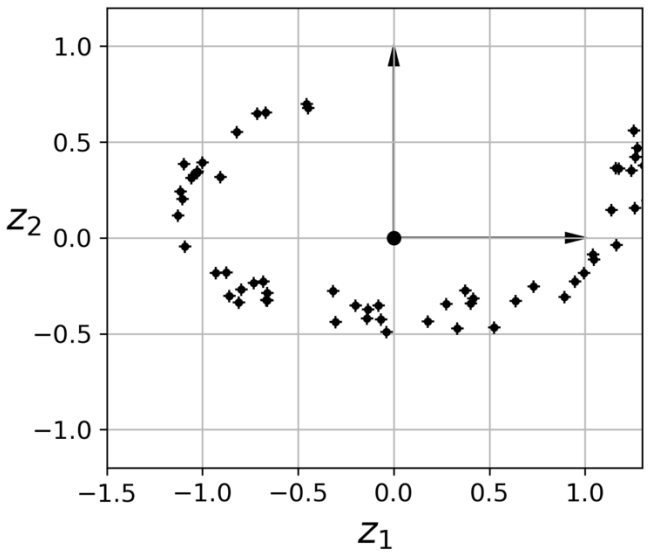
- Once you have identified all the principal components, you can reduce the dimensionality of the dataset down to d dimensions by projecting it onto the hyperplane defined by the first d principal components.
- Selecting this hyperplane ensures that the projection will preserve as much variance as possible.

$$\mathbf{X}_{d\text{-proj}} = \mathbf{X}\mathbf{W}_d$$

$\mathbf{W_d}$ is a matrix containing the first d columns of **V**

```
W2 = Vt.T[:, :2]
X2D = X_centered.dot(W2)
```





PCA in Scikit-Learn

- The following Scikit-Learn code applies PCA to reduce the dimensionality of the dataset down to two dimensions (note that it automatically takes care of centering the data):

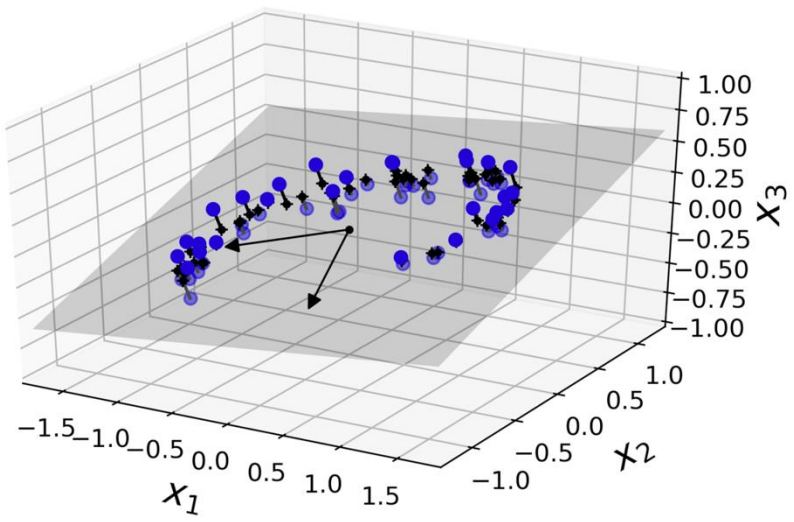
```
from sklearn.decomposition import PCA

pca = PCA(n_components = 2)
X2D = pca.fit_transform(X)
```

- After fitting the PCA transformer to the dataset, its components_ attribute holds the transpose of $\mathbf{W_d}$ (e.g., the unit vector that defines the first principal component is equal to pca.components_.T[:, 0]).

Explained Variance Ratio

- Another useful piece of information is the explained variance ratio of each principal component, available via the explained_variance_ratio_ variable.
- The ratio indicates the proportion of the dataset's variance that lies along each principal component.



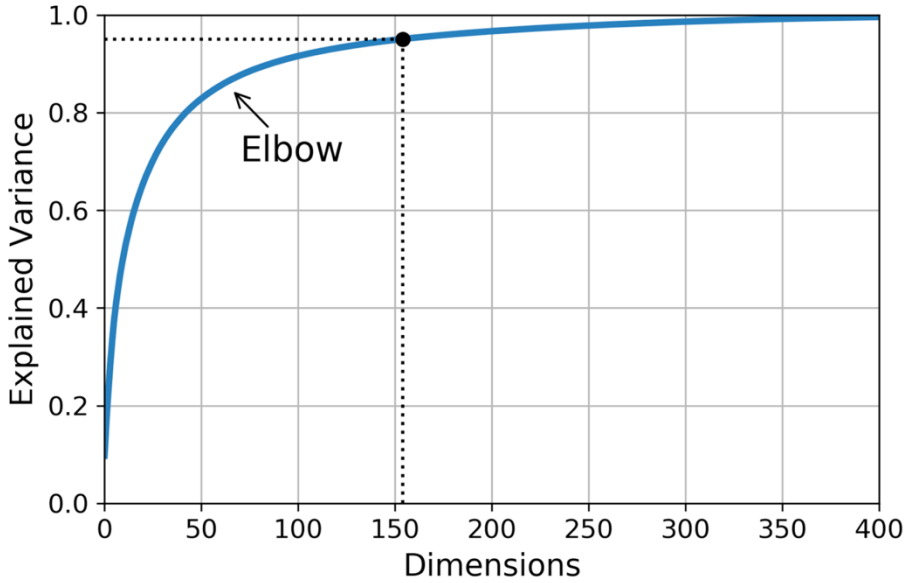

```
>>> pca.explained_variance_ratio_
array([0.84248607, 0.14631839])
```

This output tells you that 84.2% of the dataset's variance lies along the first PC, and 14.6% lies along the second PC

Explore PCA:
https://colab.research.google.com/drive/1iUafhkdpRChfptAe9Gq0uYeHdD5THXL7?usp=sharing

Choosing the Right Number of Dimensions

- One approach: plot the explained variance as a function of the number of dimensions.
- Reducing the dimensionality down to about 100 dimensions wouldn't lose too much explained variance.



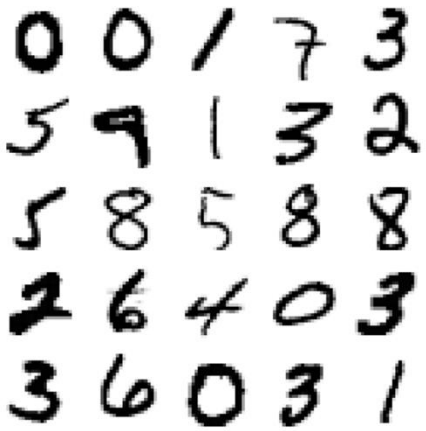
PCA for Compression

- After dimensionality reduction, the training set takes up much less space.
- Let's apply PCA to the MNIST dataset while preserving 95% of its variance, this will mean going from 784 features to 154 features.

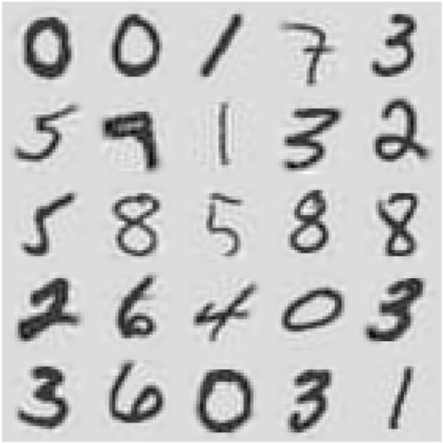
```
pca = PCA(n_components = 154)
X_reduced = pca.fit_transform(X_train)
X_recovered = pca.inverse_transform(X_reduced)
```

$$\mathbf{X}_{\text{recovered}} = \mathbf{X}_{d\text{-proj}}\mathbf{W}_d^{\mathsf{T}}$$

Original



Compressed

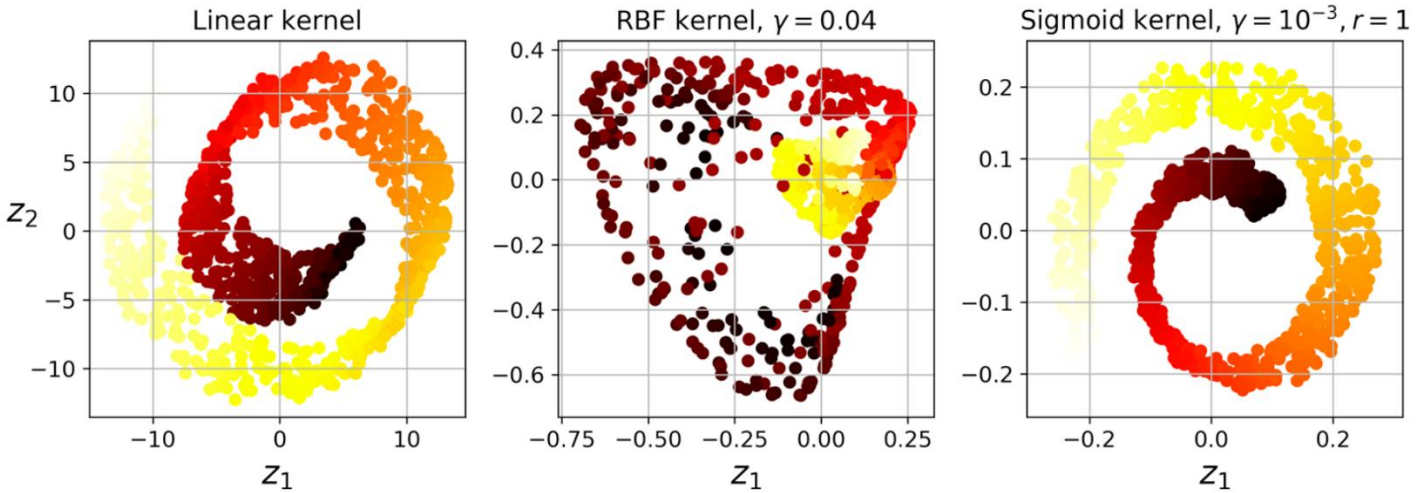


Kernel PCA

- It turns out that the same trick we used with SVMs can be applied to PCA, making it possible to perform complex nonlinear projections for dimensionality reduction. This is called Kernel PCA (kPCA).

```
from sklearn.decomposition import KernelPCA

rbf_pca = KernelPCA(n_components = 2, kernel="rbf", gamma=0.04)
X_reduced = rbf_pca.fit_transform(X)
```



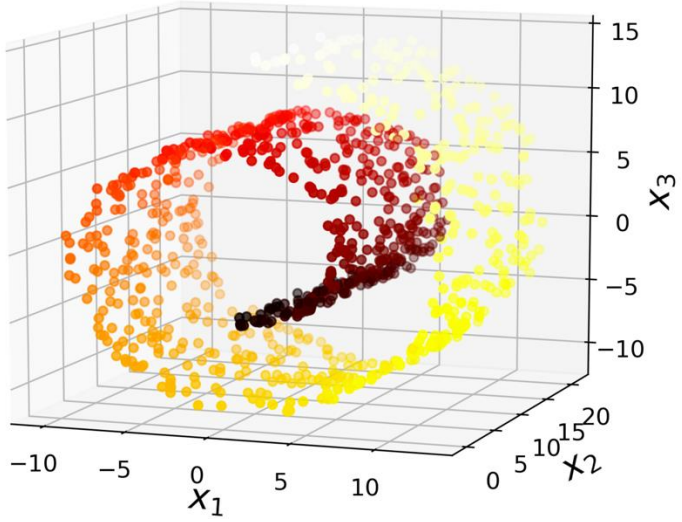
Locally Linear Embedding (LLE)

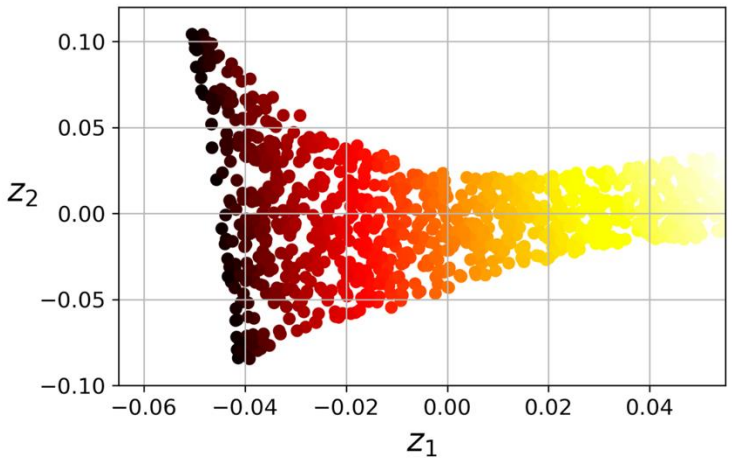
- Locally Linear Embedding (LLE) is another powerful nonlinear dimensionality reduction (NLDR) technique.
- It is a Manifold Learning technique that does not rely on projections, like PCA.
- In a nutshell, LLE works by:
  - first measuring how each training instance linearly relates to its closest neighbors
  - then looking for a low-dimensional representation of the training set where these local relationships are best preserved.

Locally Linear Embedding (LLE)

```
from sklearn.manifold import LocallyLinearEmbedding

lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10)
X_reduced = lle.fit_transform(X)
```





How LLE works?

- For each training instance $\mathbf{x}^{(i)}$ the algorithm identifies its k closest neighbors and then tries to reconstruct $\mathbf{x}^{(i)}$ as a linear function of these neighbors.
- More specifically, it finds the weights $w_{i,j}$ such that the squared distance between $\mathbf{x}^{(i)}$ and $\sum_{j=1}^{m} w_{i,j}\mathbf{x}^{(j)}$ is as small as possible:

$$\widehat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^{m} \left(\mathbf{x}^{(i)} - \sum_{j=1}^{m} w_{i,j}\mathbf{x}^{(j)} \right)^2$$

$$\text{subject to} \begin{cases} w_{i,j} = 0 & \text{if } \mathbf{x}^{(j)} \text{ is not one of the } k \text{ c.n. of } \mathbf{x}^{(i)} \\ \sum_{j=1}^{m} w_{i,j} = 1 & \text{for } i = 1, 2, \cdots, m \end{cases}$$

How LLE works?

- The second step is to map the training instances into a d-dimensional space (where d < n) while preserving these local relationships as much as possible.
- If $\mathbf{z}^{(i)}$ is the image of $\mathbf{x}^{(i)}$ in this d-dimensional space, then we want the squared distance between $\mathbf{z}^{(i)}$ and $\sum_{j=1}^{m} \widehat{w}_{i,j}\mathbf{z}^{(j)}$ to be as small as possible.

$$\widehat{\mathbf{Z}} = \underset{\mathbf{Z}}{\operatorname{argmin}} \sum_{i=1}^{m} \left(\mathbf{z}^{(i)} - \sum_{j=1}^{m} \widehat{w}_{i,j}\mathbf{z}^{(j)} \right)^2$$

- This looks very similar to the first step, but instead of keeping the instances fixed and finding the optimal weights, we are doing the reverse: keeping the weights fixed and finding the optimal position of the instances' images in the low-dimensional space.

LLE computational complexity

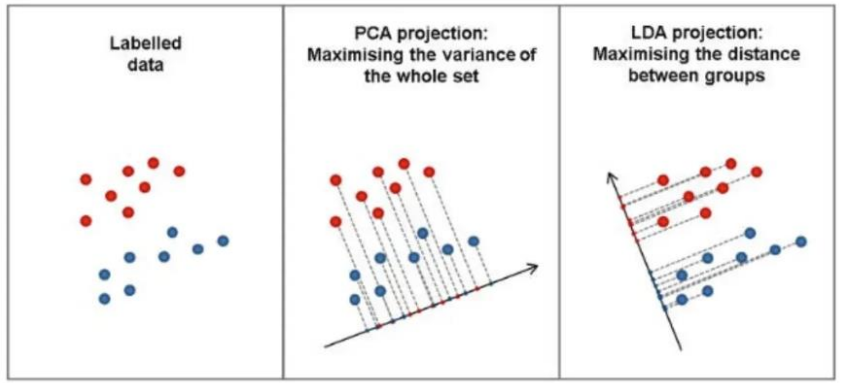
- Scikit-Learn's LLE implementation has the following computational complexity:
  - $O(m \log(m) n \log(k))$ for finding the k nearest neighbors,
  - $O(mnk^3)$ for optimizing the weights, and
  - $O(dm^2)$ for constructing the lowdimensional representations.
- Unfortunately, the m^2 in the last term makes this algorithm scale poorly to very large datasets.

Other Dimensionality Reduction Techniques

- Random Projections:
  - As its name suggests, projects the data to a lower-dimensional space using a random linear projection. This may sound crazy, but it turns out that such a random projection is actually very likely to preserve distances well.
- Multidimensional Scaling (MDS):
  - Reduces dimensionality while trying to preserve the distances between the instances.
- Isomap:
  - Creates a graph by connecting each instance to its nearest neighbors, then reduces dimensionality while trying to preserve the geodesic distances between the instances

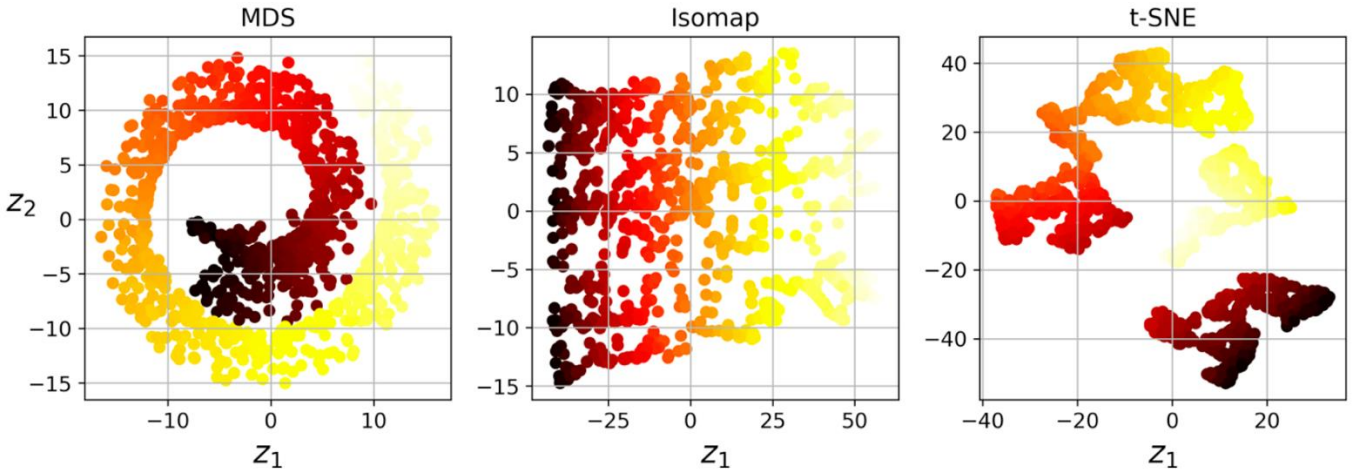
Other Dimensionality Reduction Techniques

- t-Distributed Stochastic Neighbor Embedding (t-SNE):
  - Reduces dimensionality while trying to keep similar instances close and dissimilar instances apart.
- Linear Discriminant Analysis (LDA)
  - Classification algorithm, but during training it learns the most discriminative axes between the classes, and these axes can then be used to define a hyperplane onto which to project the data.

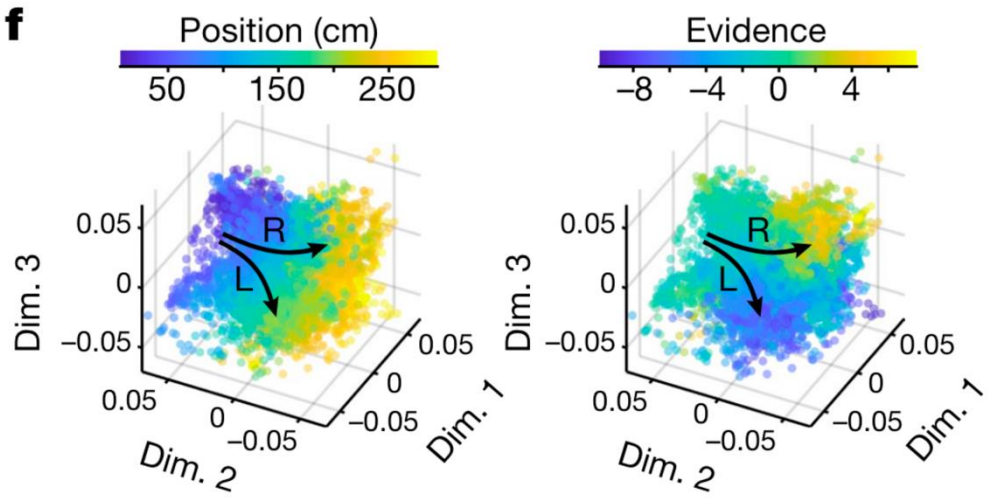


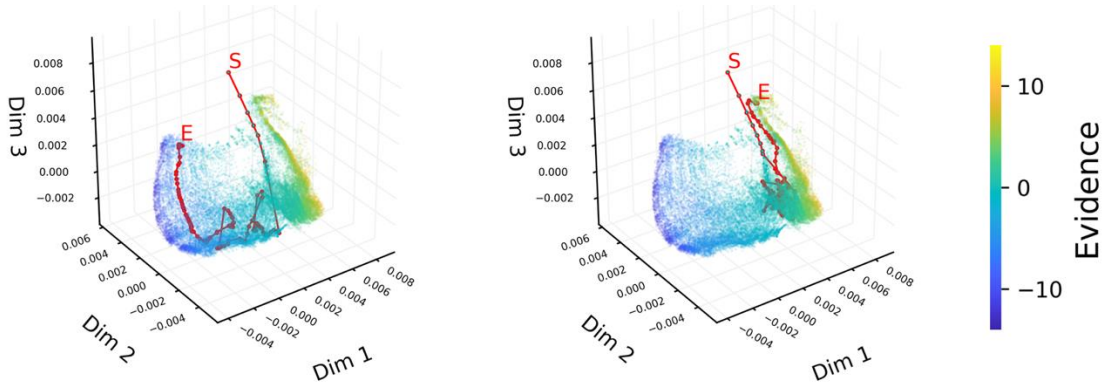
https://botpenguin.com/glossary/linear-discriminant-analysis

Other Dimensionality Reduction Techniques



Additional resources (optional): manifolds in neuroscience





https://www.nature.com/articles/s41586-021-03652-7
https://openreview.net/pdf?id=5W9so5v0OU

Next time

- Unsupervised learning, from Chapter 9, *Hands on machine learning textbook*